WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR VEHICLE SERVICE CENTRE MANAGEMENT

Program	B.Tech in Computer Science and Engineering (AI & ML)
Course & Code	Database Management Systems (DBMS)
Section	AIML Rhinos
Batch	2025 – 2029
Course Faculty	Prof. Sahithi, School of Technology
Date of Submission	October 2026
Academic Year	2026 – 2027

Submitted by:

Roll Number	Student Name	Email ID
25WU0102076	G. Keerthan Chowdary	[add your university email ID]

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for giving me the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I express my heartfelt thanks to my course faculty, Prof. Sahithi, for continuous guidance, constructive suggestions and support throughout the design and development of this project. The guidance helped me understand the practical use of core relational database concepts, including Entity-Relationship (ER) modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), SQL constraint definition, triggers, querying, transaction management and database application development using Python.

This PBL activity has strengthened my understanding of database design and transactional integrity, and gave me hands-on experience in building a practical solution for the day-to-day operations of a vehicle service centre.

UNDERTAKING

I hereby declare that the project entitled "DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR VEHICLE SERVICE CENTRE MANAGEMENT", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of my course faculty, Prof. Sahithi.

I undertake that the project report, database design, SQL scripts, application source code, results and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date: October 05, 2026	Signature: _____
Place: Hyderabad, Telangana	Name: G. Keerthan Chowdary
Roll Number: 25WU0102076	Program: B.Tech CSE (AI & ML) - AIML Rhinos
Academic Year: 2026–2027	School of Technology, Woxsen University

TABLE OF CONTENTS

Abstract & Keywords ..... **vii**
LIST OF TABLES ..... **iv**
LIST OF FIGURES ..... **v**
LIST OF ACRONYMS ..... **vi**
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM **1**
1.1 Problem Statement 1
1.2 Objectives 1
1.3 Scope and Assumptions 1
1.4 Proposed Solution Strategy 2
1.5 Functional Requirements 2
1.6 Entity-Relationship Diagram (ERD) 3
1.7 Entity and Relationship Summary 3
2. LOGICAL DATABASE DESIGN **5**
2.1 Relational Schema 5
2.2 Keys and Integrity Constraints 5
2.3 Functional Dependencies 6
2.4 Normalization up to Third Normal Form (3NF) 6
2.5 Data Dictionary 7
2.6 Implementation of Logical Design: DDL and Sample Data 9
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN **10**
3.1 Application Overview 10
3.2 Development Environment and Database Connectivity 10
3.3 Application Modules and Database Mapping 10
3.4 CRUD Operations 11
3.5 Business Rules, Validation and Error Handling 11
3.6 Reports and Database Operations 12
3.7 Application Screens 12
3.8 Testing and Results 14
4. CONCLUSION **16**
4.1 Limitations and Future Enhancements 16
5. REFERENCES **17**
6. APPENDICES **18**
APPENDIX A - COMPLETE DATABASE SCRIPT 18
APPENDIX B - APPLICATION SOURCE CODE 19
APPENDIX C - DEMONSTRATION OUTPUT AND TEST DATA 20

LIST OF TABLES

Table 1 Scope and Assumptions 1
Table 2 Proposed Solution Strategy 2
Table 3 Functional Requirements 2
Table 4 Entity and Relationship Summary 4
Table 5 Relational Schema 5
Table 6 Keys and Integrity Constraints 5
Table 7 Normalization Status up to 3NF 7
Table 8 Comprehensive Data Dictionary 7
Table 9 Database Implementation Environment 9
Table 10 Application Development Environment 10
Table 11 Application Module-to-Database Mapping 10
Table 12 CRUD Operations 11
Table 13 Business Rules, Validation and Error Handling 11
Table 14 Reports and Database Operations 12
Table 15 Test Cases and Verification Outcomes 14

LIST OF FIGURES

Figure 1 Entity-Relationship Diagram of the Proposed System 3
Figure 2 System Architecture of the Application 10
Figure 3 Booking, Check-in and Mechanic Scheduling Output 12
Figure 4 Spare Parts Issue and Invoice Generation Output 13
Figure 5 Operational Reports Output 13
Figure 6 Workshop Analytics Charts 13

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce-Codd Normal Form
CLI	Command Line Interface
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
FK	Foreign Key
GST	Goods and Services Tax
PK	Primary Key
RDBMS	Relational Database Management System
SQL	Structured Query Language

ABSTRACT

Vehicle service centres handle a continuous flow of customers, vehicles, repair tasks, mechanics, spare parts and invoices. When these are recorded on paper job cards, registers and isolated spreadsheets, repair history is lost, mechanics are scheduled into conflicting time slots, spare parts run out unnoticed, and invoices contain avoidable errors. The objective of this project is to design, normalize and implement a Relational Database Management System (RDBMS) for a multi-brand vehicle service centre.

The conceptual model contains ten interrelated entities: customers, vehicles, service bookings, job cards, mechanics, tasks, assignments, spare parts, part usage and invoices. The design is normalized to Third Normal Form (3NF) to remove redundancy and update anomalies. Critical business rules, namely one active booking per vehicle slot, one job card per booking, no overlapping mechanic assignments, no issue of parts beyond available stock, positive quantities and one invoice per job card, are enforced through declarative constraints and triggers. A Python application built on the standard sqlite3 interface supports registration, booking, check-in, task scheduling, parts issue with automatic stock deduction, invoice generation with GST, and operational reports. A test suite of fifteen cases, including negative integrity tests, was executed and all cases behaved as specified.

KEYWORDS

Relational Database, ER Diagram, Normalization, SQL, Python, Vehicle Service Centre, Triggers, Integrity Constraints

CHAPTER 1: PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Multi-brand vehicle service centres run many jobs in parallel: customers book slots, vehicles are checked in, mechanics with different specializations work on repair tasks, spare parts are drawn from a shared store, and a final invoice is raised when the job is complete. In many workshops this information still lives on paper job cards, physical registers and separate spreadsheets. Nothing links a customer to a vehicle, a vehicle to its repair history, or a repair to the parts and labor consumed.

This fragmentation causes several recurring problems. Repair progress is hard to track, so history is misplaced and customers are not kept informed. Without automated schedule management, workshop heads cannot assign mechanics by availability, and two jobs are allocated to the same mechanic in the same time slot, which delays deliveries. Spare parts are issued without updating stock, causing unexpected stock-outs and incorrect invoices. Billing depends on manually adding labor and parts, so totals are error-prone and past prices cannot be verified. There is therefore a clear need for a centralized, normalized relational database that enforces integrity rules, keeps inventory accurate and automates billing.

1.2 Objectives

The primary objectives of this project are:

1. Design a conceptual Entity-Relationship (ER) model covering the ten core entities of a vehicle service centre, with precise cardinalities.
2. Derive the relational schema and normalize it to Third Normal Form (3NF) using functional dependencies, removing insertion, deletion and update anomalies.
3. Implement DDL constraints and triggers that enforce the key business rules: no duplicate slot bookings, one job card per booking, no overlapping mechanic assignments, no issue of parts beyond stock, and positive quantities.
4. Develop SQL queries and reports for low stock, mechanic workload, unpaid invoices, monthly revenue and job card status.
5. Build a Python application, connected through the standard database driver, that supports the complete workflow from registration to invoice with clear error handling.

1.3 Scope and Assumptions

The scope boundaries and technical assumptions of this implementation are summarized in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	Customer and vehicle registration; service slot booking; check-in and job card creation; mechanic and task catalogs; task assignment with overlap prevention; spare parts catalog and stock tracking; parts issue to job cards; invoice generation with labor, parts and 18% GST; payment status; operational reports.
Out of Scope / Exclusions	Online customer portal; live payment gateway; SMS or email notifications; vehicle diagnostics hardware integration; multi-branch consolidation; user authentication and role-based login.
Assumptions	Each booking is for one vehicle in one slot; a booking produces at most one job card; labor is charged at the flat rate of each task assigned; parts are billed at the unit price in force when issued; GST is fixed at 18%; one invoice is raised per job card.

1.4 Proposed Solution Strategy

The solution replaces paper records and spreadsheets with a single relational database and a Python application that works on it. Business rules are placed inside the database wherever possible so that they apply to every route into the data. Table 2 outlines the strategy.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Service Advisor, Service Mechanic, Inventory Controller and Workshop Manager.
Major Application Modules	Customer and Vehicle Registration; Booking and Job Card Management; Mechanic Scheduling; Parts and Inventory; Billing; Reports.
Proposed Workflow	Advisor registers the customer and vehicle and books a slot -> vehicle is checked in and a job card is opened -> tasks are assigned to mechanics with non-overlapping time windows -> parts are issued from stock against the job card -> invoice is generated from labor and parts with GST -> job card is closed and payment is recorded.
Important Business Rules	One booking per vehicle, date and slot; one job card per booking; no overlapping assignments for a mechanic; parts cannot be issued beyond stock; quantities and prices must be positive; one invoice per job card.
Expected Reports / Outputs	Job card status; low-stock parts; mechanic workload in hours; unpaid invoices with customer names; monthly revenue.

1.5 Functional Requirements

The functional requirements of the system are listed and prioritized in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall store customer profiles and the vehicles registered to each customer.	High / Core Master Data
FR-02	The system shall record service bookings with date and slot and reject duplicate bookings for the same vehicle and slot.	High / Integrity Rule

Req. ID	Functional Requirement	Remarks / Priority
FR-03	The system shall open exactly one job card for a booking when the vehicle is checked in.	High / Core Workflow
FR-04	The system shall maintain mechanic and task catalogs, including specialization and labor rate.	Medium / Master Data
FR-05	The system shall assign tasks to mechanics with start and end times and reject overlapping assignments for the same mechanic.	High / Critical Integrity Rule
FR-06	The system shall maintain the spare parts catalog with unit price, stock quantity and reorder level.	High / Master Data
FR-07	The system shall issue parts to a job card, verify stock, deduct stock automatically and record the price billed.	High / Inventory Integrity
FR-08	The system shall generate one invoice per job card from labor and parts totals plus 18% GST.	High / Financial Accuracy
FR-09	The system shall close the job card, complete the booking and record the payment status once the invoice is raised.	Medium / Workflow
FR-10	The system shall provide reports for job cards, low stock, mechanic workload, unpaid invoices and monthly revenue.	High / Business Analytics

1.6 Entity-Relationship Diagram (ERD)

The conceptual design is shown in Figure 1. The ER diagram models ten entities with their primary keys, foreign keys and cardinalities. A customer owns many vehicles, a vehicle has many bookings, and each booking opens at most one job card. A job card is the hub of the workflow: it has many task assignments, consumes many parts, and is billed by exactly one invoice. Assignment and Part_Usage are associative entities that resolve the many-to-many relationships between job cards and tasks or mechanics, and between job cards and spare parts.

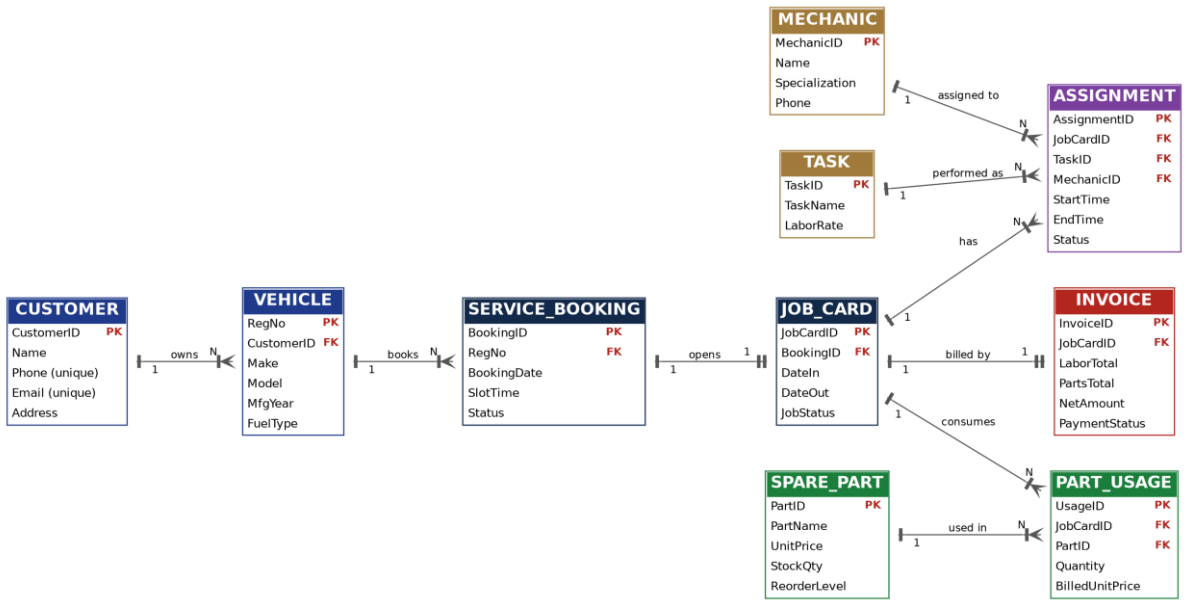


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 documents the entity catalog with attributes, cardinalities and business rules.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
CUSTOMER	CustomerID (PK), Name, Phone, Email, Address	Customer master; Phone and Email are unique.
VEHICLE	RegNo (PK), CustomerID (FK), Make, Model, MfgYear, FuelType 1:N with CUSTOMER	Registered vehicles; year and fuel type are validated.
SERVICE_BOOKING	BookingID (PK), RegNo (FK), BookingDate, SlotTime, Status 1:N with VEHICLE	Slot reservation; UNIQUE (RegNo, BookingDate, SlotTime).
JOB_CARD	JobCardID (PK), BookingID (FK), DateIn, DateOut, JobStatus 1:1 with SERVICE_BOOKING	Work order opened at check-in; DateOut >= DateIn.
MECHANIC	MechanicID (PK), Name, Specialization, Phone	Workshop staff master; Phone is unique.
TASK	TaskID (PK), TaskName, LaborRate	Catalog of repair tasks; LaborRate >= 0.
ASSIGNMENT	AssignmentID (PK), JobCardID, TaskID, MechanicID (FKs), StartTime, EndTime, Status M:N JOB_CARD-TASK-MECHANIC	Associative entity; EndTime > StartTime; trigger blocks overlapping slots.
SPARE_PART	PartID (PK), PartName, UnitPrice, StockQty, ReorderLevel	Inventory master; UnitPrice > 0, StockQty >= 0.
PART_USAGE	UsageID (PK), JobCardID (FK), PartID (FK), Quantity, BilledUnitPrice M:N JOB_CARD-SPARE_PART	Parts issued to a job; Quantity > 0; trigger checks and deducts stock.
INVOICE	InvoiceID (PK), JobCardID (FK), LaborTotal, PartsTotal, NetAmount, PaymentStatus 1:1 with JOB_CARD	Final bill; one invoice per job card.

CHAPTER 2: LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was mapped to a relational schema of ten relations. Primary and foreign keys are defined for every relation to preserve referential integrity. Table 5 lists the relations.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	Customer	Stores customer profiles and contact details.
2	Vehicle	Stores vehicles registered to customers, with make, model, year and fuel type.
3	Service_Booking	Stores service slot reservations for vehicles.
4	Job_Card	Stores the work order opened when a booked vehicle is checked in.
5	Mechanic	Stores workshop mechanics and their specializations.
6	Task	Stores the catalog of repair tasks and labor rates.
7	Assignment	Associative relation linking job cards, tasks and mechanics with working times.
8	Spare_Part	Stores spare parts with price, stock quantity and reorder level.
9	Part_Usage	Associative relation recording parts issued to job cards with the price billed.
10	Invoice	Stores the final bill for a job card with labor, parts and net amounts.

2.2 Keys and Integrity Constraints

Declarative and procedural constraints guarantee consistency and prevent illegal data states. Table 6 maps the constraints to each relation.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Customer	PK / UNIQUE / NOT NULL	CustomerID / Phone, Email / Name	Unique customer identity; unique phone and email.
Vehicle	PK / FK / CHECK	RegNo / CustomerID / MfgYear, FuelType	Vehicle belongs to a valid customer; year 1990-2030; fuel in Petrol, Diesel, CNG, Electric.
Service_Booking	PK / FK / UNIQUE / CHECK	BookingID / RegNo / (RegNo, BookingDate, SlotTime) / Status	No duplicate slot for a vehicle; status in Booked, Checked-In, Completed, Cancelled.
Job_Card	PK / FK / UNIQUE / CHECK	JobCardID / BookingID / BookingID / DateOut, JobStatus	One job card per booking; DateOut >= DateIn; status in Open, In Progress, Closed.
Mechanic	PK / UNIQUE / NOT NULL	MechanicID / Phone / Name	Unique mechanic phone; mandatory name.
Task	PK / UNIQUE / CHECK	TaskID / TaskName / LaborRate	Unique task name; non-negative labor rate.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
Assignment	PK / FK / CHECK / Trigger	AssignmentID / JobCardID, TaskID, MechanicID / EndTime / trg_no_overlap	EndTime after StartTime; a mechanic cannot hold overlapping slots.
Spare_Part	PK / UNIQUE / CHECK	PartID / PartName / UnitPrice, StockQty	Unique part name; price > 0; stock >= 0.
Part_Usage	PK / FK / CHECK / Triggers	UsageID / JobCardID, PartID / Quantity / trg_check_stock, trg_deduct_stock	Quantity > 0; parts cannot exceed stock; stock reduced after issue.
Invoice	PK / FK / UNIQUE	InvoiceID / JobCardID / JobCardID	One invoice per job card.

2.3 Functional Dependencies

The non-trivial functional dependencies (FDs) identified for each relation are given below. They justify the normalization decisions in Section 2.4.

1. Customer: CustomerID -> {Name, Phone, Email, Address}; Phone -> {CustomerID, Name, Email, Address}; Email -> {CustomerID, Name, Phone, Address}.
2. Vehicle: RegNo -> {CustomerID, Make, Model, MfgYear, FuelType}.
3. Service_Booking: BookingID -> {RegNo, BookingDate, SlotTime, Status}; {RegNo, BookingDate, SlotTime} -> {BookingID, Status}.
4. Job_Card: JobCardID -> {BookingID, DateIn, DateOut, JobStatus}; BookingID -> {JobCardID, DateIn, DateOut, JobStatus}.
5. Mechanic: MechanicID -> {Name, Specialization, Phone}; Phone -> {MechanicID, Name, Specialization}.
6. Task: TaskID -> {TaskName, LaborRate}; TaskName -> {TaskID, LaborRate}.
7. Assignment: AssignmentID -> {JobCardID, TaskID, MechanicID, StartTime, EndTime, Status}.
8. Spare_Part: PartID -> {PartName, UnitPrice, StockQty, ReorderLevel}; PartName -> {PartID, UnitPrice, StockQty, ReorderLevel}.
9. Part_Usage: UsageID -> {JobCardID, PartID, Quantity, BilledUnitPrice}.
10. Invoice: InvoiceID -> {JobCardID, LaborTotal, PartsTotal, NetAmount, PaymentStatus}; JobCardID -> {InvoiceID, LaborTotal, PartsTotal, NetAmount, PaymentStatus}.

2.4 Normalization up to Third Normal Form (3NF)

Every relation was checked against the standard normal forms.

- 1NF: all attributes hold atomic values and there are no repeating groups. Repeating tasks and parts of a job are stored as rows in Assignment and Part_Usage.
- 2NF: the relations are in 1NF and no non-prime attribute depends on part of a candidate key. Customer details depend on CustomerID, vehicle details on RegNo, part price on PartID and task rate on TaskID, so each is held in its own relation.

- 3NF: the relations are in 2NF and no non-prime attribute is transitively dependent on a key. Bookings store only RegNo, not owner details; job cards store only BookingID.
- BCNF: for every non-trivial FD X -> Y listed in Section 2.3, X is a superkey, so all ten relations also satisfy BCNF.

Table 7 summarizes the result.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
Customer	Yes	Yes	Yes	Yes	Candidate keys: CustomerID, Phone, Email.
Vehicle	Yes	Yes	Yes	Yes	Owner details kept in Customer; only CustomerID stored.
Service_Booking	Yes	Yes	Yes	Yes	Vehicle and owner data not repeated; slot is a candidate key.
Job_Card	Yes	Yes	Yes	Yes	1:1 with booking; BookingID is a candidate key.
Mechanic	Yes	Yes	Yes	Yes	Candidate keys: MechanicID, Phone.
Task	Yes	Yes	Yes	Yes	Labor rate depends only on TaskID.
Assignment	Yes	Yes	Yes	Yes	Resolves M:N between job cards, tasks and mechanics.
Spare_Part	Yes	Yes	Yes	Yes	Price and stock depend only on PartID.
Part_Usage	Yes	Yes	Yes	Yes	Resolves M:N between job cards and parts; BilledUnitPrice is a historical snapshot, not derived from PartID.
Invoice	Yes	Yes	Yes*	Yes*	*NetAmount is computable from LaborTotal and PartsTotal. It is stored deliberately as an immutable billing snapshot (a controlled, documented denormalization).

2.5 Data Dictionary

Table 8 gives the data dictionary for all attributes as implemented in the database.

Table 8. Comprehensive Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
Customer	CustomerID	INTEGER	PK, AUTOINCREMENT	Unique customer identifier.
Customer	Name	TEXT	NOT NULL	Customer full name.
Customer	Phone	TEXT	NOT NULL, UNIQUE	Primary contact number.
Customer	Email	TEXT	UNIQUE	Contact email address.
Customer	Address	TEXT	-	Residential or billing address.
Vehicle	RegNo	TEXT	PK	Vehicle registration number.
Vehicle	CustomerID	INTEGER	FK, NOT NULL	Owner; ON DELETE CASCADE.
Vehicle	Make	TEXT	NOT NULL	Manufacturer name.
Vehicle	Model	TEXT	NOT NULL	Model name.
Vehicle	MfgYear	INTEGER	CHECK 1990 to 2030	Year of manufacture.
Vehicle	FuelType	TEXT	CHECK IN list	Petrol, Diesel, CNG or Electric.
Service_Booking	BookingID	INTEGER	PK, AUTOINCREMENT	Unique booking identifier.
Service_Booking	RegNo	TEXT	FK, NOT NULL	Vehicle being booked.
Service_Booking	BookingDate	TEXT	NOT NULL	Date of the service slot.
Service_Booking	SlotTime	TEXT	NOT NULL	Time of the service slot.

Table	Attribute	Data Type	Key / Constraint	Description
Service_Booking	Status	TEXT	DEFAULT 'Booked', CHECK	Booked, Checked-In, Completed or Cancelled.
Job_Card	JobCardID	INTEGER	PK, AUTOINCREMENT	Unique job card identifier.
Job_Card	BookingID	INTEGER	FK, NOT NULL, UNIQUE	Booking this job card belongs to.
Job_Card	DateIn	TEXT	NOT NULL	Date the vehicle was checked in.
Job_Card	DateOut	TEXT	CHECK >= DateIn	Date the job was closed.
Job_Card	JobStatus	TEXT	DEFAULT 'Open', CHECK	Open, In Progress or Closed.
Mechanic	MechanicID	INTEGER	PK, AUTOINCREMENT	Unique mechanic identifier.
Mechanic	Name	TEXT	NOT NULL	Mechanic full name.
Mechanic	Specialization	TEXT	-	Area of expertise.
Mechanic	Phone	TEXT	UNIQUE	Contact number.
Task	TaskID	INTEGER	PK, AUTOINCREMENT	Unique task identifier.
Task	TaskName	TEXT	NOT NULL, UNIQUE	Name of the repair task.
Task	LaborRate	REAL	CHECK >= 0	Labor charge for the task.
Assignment	AssignmentID	INTEGER	PK, AUTOINCREMENT	Unique assignment identifier.
Assignment	JobCardID	INTEGER	FK, NOT NULL	Job card the task belongs to.
Assignment	TaskID	INTEGER	FK, NOT NULL	Task to be performed.
Assignment	MechanicID	INTEGER	FK, NOT NULL	Mechanic performing the task.
Assignment	StartTime	TEXT	NOT NULL	Scheduled start date and time.
Assignment	EndTime	TEXT	CHECK > StartTime	Scheduled end date and time.
Assignment	Status	TEXT	DEFAULT 'Assigned'	Progress of the task.
Spare_Part	PartID	INTEGER	PK, AUTOINCREMENT	Unique part identifier.
Spare_Part	PartName	TEXT	NOT NULL, UNIQUE	Name of the spare part.
Spare_Part	UnitPrice	REAL	CHECK > 0	Current selling price per unit.
Spare_Part	StockQty	INTEGER	NOT NULL, DEFAULT 0, CHECK >= 0	Units currently in stock.
Spare_Part	ReorderLevel	INTEGER	DEFAULT 5	Stock level that triggers reordering.
Part_Usage	UsageID	INTEGER	PK, AUTOINCREMENT	Unique usage identifier.
Part_Usage	JobCardID	INTEGER	FK, NOT NULL	Job card the part was issued to.
Part_Usage	PartID	INTEGER	FK, NOT NULL	Part issued.
Part_Usage	Quantity	INTEGER	NOT NULL, CHECK > 0	Units issued.
Part_Usage	BilledUnitPrice	REAL	NOT NULL	Unit price locked at the time of issue.
Invoice	InvoiceID	INTEGER	PK, AUTOINCREMENT	Unique invoice identifier.
Invoice	JobCardID	INTEGER	FK, NOT NULL, UNIQUE	Job card being billed.
Invoice	LaborTotal	REAL	DEFAULT 0	Sum of task labor charges.
Invoice	PartsTotal	REAL	DEFAULT 0	Sum of quantity x billed price.
Invoice	NetAmount	REAL	DEFAULT 0	(Labor + parts) plus 18% GST.
Invoice	PaymentStatus	TEXT	DEFAULT 'Unpaid'	Unpaid or Paid.

2.6 Implementation of Logical Design: DDL and Sample Data

The database implementation details are summarized in Table 9.

Table 9. Database Implementation Environment

Item	Details
DBMS Engine	SQLite 3 relational engine (embedded, ANSI SQL compliant) with PRAGMA foreign_keys = ON
Number of Relational Tables	10 tables and 3 triggers
Reference Data Loaded by the Application	2 mechanics, 3 repair tasks, 3 spare parts
Demonstration Scenario Data	1 customer, 1 vehicle, 1 booking, 1 job card, 2 task assignments, 2 part issues, 1 invoice
Test Suite	15 test cases covering valid operations and negative integrity tests

Representative DDL statements for the job card, the overlap trigger and the stock triggers are shown below. The complete script is given in Appendix A.

```
CREATE TABLE Job_Card (
  JobCardID INTEGER PRIMARY KEY AUTOINCREMENT,
  BookingID INTEGER NOT NULL UNIQUE REFERENCES Service_Booking(BookingID),
  DateIn TEXT NOT NULL, DateOut TEXT,
  JobStatus TEXT DEFAULT 'Open' CHECK (JobStatus IN ('Open','In Progress','Closed')),
  CHECK (DateOut IS NULL OR DateOut >= DateIn));

CREATE TRIGGER trg_no_overlap BEFORE INSERT ON Assignment
WHEN NEW.EndTime IS NOT NULL AND EXISTS (
  SELECT 1 FROM Assignment a
  WHERE a.MechanicID = NEW.MechanicID AND a.EndTime IS NOT NULL
    AND NEW.StartTime < a.EndTime AND NEW.EndTime > a.StartTime)
BEGIN SELECT RAISE(ABORT, 'Mechanic already booked in that time slot'); END;

CREATE TRIGGER trg_check_stock BEFORE INSERT ON Part_Usage
WHEN (SELECT StockQty FROM Spare_Part WHERE PartID = NEW.PartID) < NEW.Quantity
BEGIN SELECT RAISE(ABORT, 'Insufficient stock'); END;

CREATE TRIGGER trg_deduct_stock AFTER INSERT ON Part_Usage
BEGIN UPDATE Spare_Part SET StockQty = StockQty - NEW.Quantity
      WHERE PartID = NEW.PartID; END;
```

CHAPTER 3: APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The front-end application is developed in Python to demonstrate how the normalized database is used in daily operations. It is a menu-driven terminal application that lets a service advisor register customers and vehicles, book service slots and check vehicles in; lets the workshop schedule mechanics; lets the inventory controller issue parts with automatic stock updates; produces invoices; and lets the workshop manager view operational reports. Every database change is committed only when it succeeds, and any rejected statement is rolled back with a clear message taken from the database.

3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming Language	Python 3
DBMS & Storage	SQLite 3 embedded relational database
IDE / Tooling	Visual Studio Code and command line
Database Connector / Driver	Python Standard Database API (DB-API 2.0) sqlite3 module
Operating Environment	Cross-platform (Windows, Linux, macOS); standard library only
Security & Integrity Protocols	PRAGMA foreign_keys = ON; parameterized SQL bindings; explicit commit and rollback

3.3 Application Modules and Database Mapping

Figure 2 shows the architecture of the application, and Table 11 maps each module to the tables and operations it uses.

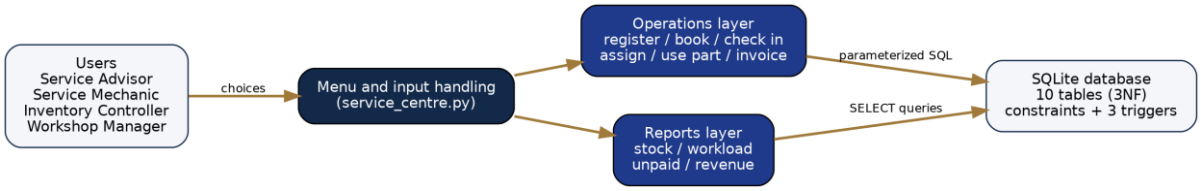


Figure 2. System Architecture of the Application

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Customer & Vehicle	Register customers and their vehicles.	Customer, Vehicle	INSERT with UNIQUE, CHECK and FK validation.
Booking & Job Cards	Book slots; check in and open job cards.	Service_Booking, Job_Card	INSERT booking; UPDATE status to Checked-In; INSERT job card.
Mechanic Scheduling	Assign tasks to mechanics with time windows.	Assignment, Task, Mechanic	INSERT assignment; trigger rejects overlaps.

Module / Screen	Purpose	Tables Used	Main Operations
Parts & Inventory	Issue parts and keep stock accurate.	Part_Usage, Spare_Part	INSERT usage with price copied from Spare_Part; triggers check and deduct stock.
Billing	Compute and store the invoice.	Invoice, Assignment, Task, Part_Usage, Job_Card, Service_Booking	SELECT SUM labor and parts; INSERT invoice; UPDATE job card and booking.
Reports	Operational reports for managers.	All tables through JOINs	SELECT with JOIN, GROUP BY and aggregates.

3.4 CRUD Operations

The application supports Create, Read and Update operations on all operational data. Records are not physically deleted during normal work: bookings and job cards move through status values so that the service history is preserved. Table 12 details these operations.

Table 12. CRUD Operations

Operation	Application Feature	Table(s) Affected	Evidence
Create	Registration, booking, check-in, task assignment, parts issue, invoicing	Customer, Vehicle, Service_Booking, Job_Card, Assignment, Part_Usage, Invoice	INSERT INTO Service_Booking (RegNo, BookingDate, SlotTime) VALUES (?,?,?)
Read / Search	Job card status and all reports	All tables; joins across booking, vehicle and customer	SELECT ... FROM Spare_Part WHERE StockQty <= ReorderLevel
Update	Check-in, stock deduction, job closure, payment	Service_Booking, Spare_Part, Job_Card, Invoice	UPDATE Invoice SET PaymentStatus = 'Paid' WHERE InvoiceID = ?
Controlled Removal	Status change instead of deletion; cascade only when a customer is removed	Service_Booking, Job_Card, Vehicle	Status moves Booked -> Checked-In -> Completed; vehicles cascade from Customer

3.5 Business Rules, Validation and Error Handling

Table 13 shows how key business rules are enforced by layers of database constraints, triggers and application logic. Every write goes through a single helper that executes the statement, commits on success, and on failure rolls back and prints the database error.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
One booking per vehicle slot	UNIQUE (RegNo, BookingDate, SlotTime) on Service_Booking.	UNIQUE constraint failed: Service_Booking.RegNo, ...
One job card per booking	UNIQUE on Job_Card.BookingID.	UNIQUE constraint failed: Job_Card.BookingID
No overlapping mechanic slots	BEFORE INSERT trigger trg_no_overlap on Assignment.	Mechanic already booked in that time slot
Stock cannot go negative	BEFORE INSERT trigger trg_check_stock; CHECK StockQty >= 0.	Insufficient stock
Positive quantities and prices	CHECK (Quantity > 0), CHECK (UnitPrice > 0).	CHECK constraint failed: Quantity > 0

Rule / Validation	How Implemented	Evidence / Message
Valid dates and times	CHECK (EndTime > StartTime), CHECK (DateOut >= DateIn).	CHECK constraint failed: EndTime IS NULL OR EndTime > StartTime
Referential integrity	FOREIGN KEY constraints with PRAGMA foreign_keys = ON.	FOREIGN KEY constraint failed

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Job Card Status	JOIN of Job_Card, Service_Booking and Vehicle.	Job card, registration number, status and dates.
Low Stock Parts	WHERE StockQty <= ReorderLevel.	Parts that need reordering.
Mechanic Workload	JOIN with GROUP BY; SUM of (EndTime - StartTime) in hours.	Tasks and hours per mechanic.
Unpaid Invoices	Five-table JOIN with PaymentStatus = 'Unpaid'.	Invoice, customer name and amount due.
Monthly Revenue	GROUP BY month of DateOut with SUM(NetAmount).	Revenue per month for closed jobs.
Invoice Generation	Subqueries with SUM over Assignment x Task and Part_Usage; GST applied.	Labor, parts and net amount stored in Invoice.

3.7 Application Screens

Figures 3 to 6 show the output of the application during the scripted demonstration. The scenario follows one customer, Ravi Kumar, who books his Honda City for 12 October 2026. Lines marked [OK] succeeded and lines marked [BLOCKED] were rejected by the database.

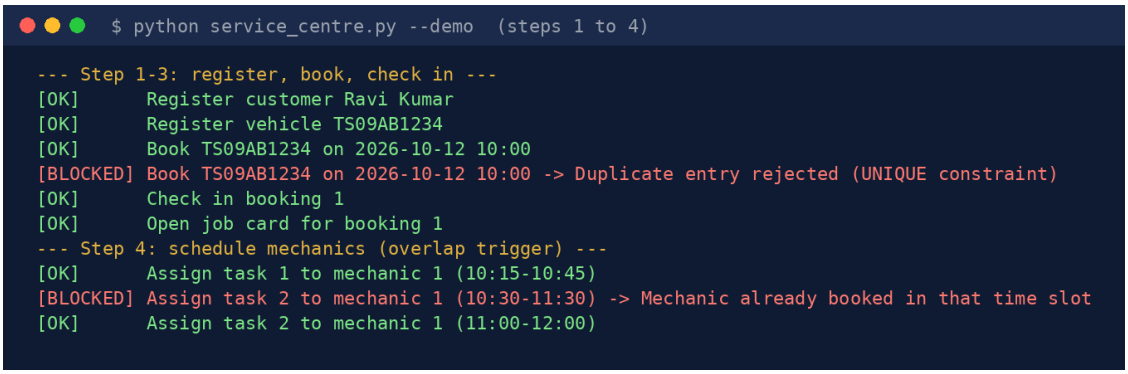


Figure 3. Booking, Check-in and Mechanic Scheduling Output

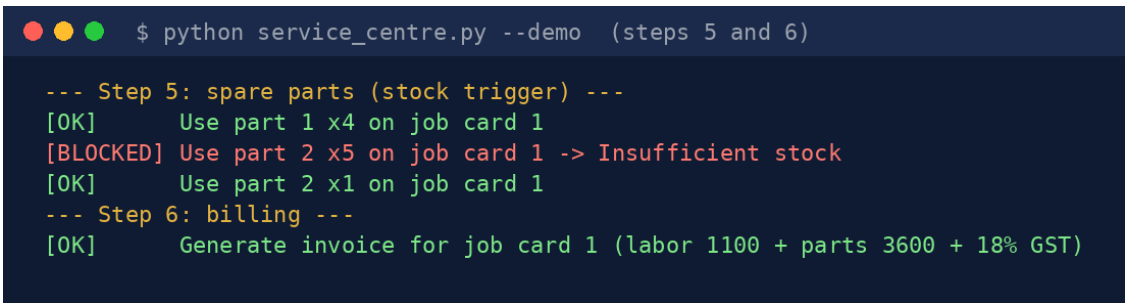


Figure 4. Spare Parts Issue and Invoice Generation Output

```
$ python service_centre.py --demo  (step 7: reports)

== Job cards ==
JobCardID  RegNo       JobStatus  DateIn      DateOut
---------  ----------  ---------  ----------  ----------
1          TS09AB1234  Closed     2026-10-12  2026-10-12
== Low stock parts ==
PartName       StockQty  ReorderLevel
-------------  --------  ------------
Brake Pad Set  2         5
== Mechanic workload ==
Name        Tasks  Hours
----------  -----  -----
Imran Khan  2      1.50
== Unpaid invoices ==
InvoiceID  Name        NetAmount
---------  ----------  ---------
1          Ravi Kumar  5546.00
== Monthly revenue ==
Month    Revenue
-------  -------
2026-10  5546.00
[OK]      Mark invoice 1 paid
== Unpaid invoices ==
InvoiceID  Name  NetAmount
---------  ----  ---------
(no rows)
```

Figure 5. Operational Reports Output

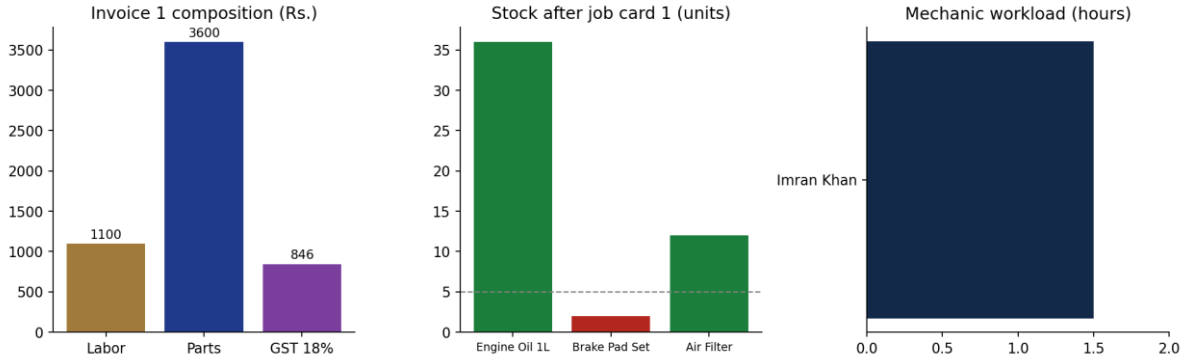


Figure 6. Workshop Analytics Charts

3.8 Testing and Results

Verification covered both valid workflows and negative tests that try to break the integrity rules. Table 15 lists the test cases with the actual messages returned by the database.

Table 15. Test Cases and Verification Outcomes

ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Vehicle registration	Register TS09AB1234 (Honda City, Petrol) for customer 1	Row inserted	Vehicle TS09AB1234 registered	PASSED
TC-02	Unique phone (negative)	Register a second customer with phone 9876543210	UNIQUE constraint rejects the row	UNIQUE constraint failed: Customer.Phone	PASSED

ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-03	Fuel type check (negative)	Insert vehicle with FuelType = 'Steam'	CHECK constraint rejects the row	CHECK constraint failed: FuelType IN ('Petrol','Diesel','CNG','Electric')	PASSED
TC-04	Referential integrity (negative)	Insert vehicle for non-existent customer 99	Foreign key rejects the row	FOREIGN KEY constraint failed	PASSED
TC-05	Duplicate booking (negative)	Book TS09AB1234 again for 2026-10-12 at 10:00	UNIQUE (RegNo, BookingDate, SlotTime) rejects the row	UNIQUE constraint failed: Service_Booking.RegNo, Service_Booking.BookingDate, Service_Booking.SlotTime	PASSED
TC-06	One job card per booking (negative)	Open a second job card for booking 1	UNIQUE on BookingID rejects the row	UNIQUE constraint failed: Job_Card.BookingID	PASSED
TC-07	Task time order (negative)	Assign a task with EndTime earlier than StartTime	CHECK (EndTime > StartTime) rejects the row	CHECK constraint failed: EndTime IS NULL OR EndTime > StartTime	PASSED
TC-08	Mechanic overlap (negative)	Assign mechanic 1 to 10:30-11:30 (overlaps 10:15-10:45)	trg_no_overlap aborts the insert	Mechanic already booked in that time slot	PASSED
TC-09	Mechanic scheduling	Assign mechanic 1 to 11:00-12:00 (no overlap)	Row inserted	Assignment inserted	PASSED
TC-10	Out-of-stock part (negative)	Issue 5 Brake Pad Sets (stock = 3)	trg_check_stock aborts the insert	Insufficient stock	PASSED
TC-11	Positive quantity (negative)	Issue a part with Quantity = -2	CHECK (Quantity > 0) rejects the row	CHECK constraint failed: Quantity > 0	PASSED
TC-12	Stock deduction	Issue 4 Engine Oil and 1 Brake Pad Set	Stock falls from 40 to 36 and from 3 to 2	Stock now: Engine Oil 1L = 36, Brake Pad Set = 2, Air Filter = 12	PASSED
TC-13	Invoice calculation	Generate invoice for job card 1	(1100 + 3600) x 1.18 = 5546.00	Labor 1100.00, Parts 3600.00, Net 5546.00	PASSED
TC-14	Duplicate invoice (negative)	Insert a second invoice for job card 1	UNIQUE on JobCardID rejects the row	UNIQUE constraint failed: Invoice.JobCardID	PASSED
TC-15	Date order on job card (negative)	Update job card DateOut to a date before DateIn	CHECK (DateOut >= DateIn) rejects the update	CHECK constraint failed: DateOut IS NULL OR DateOut >= DateIn	PASSED

All fifteen test cases behaved as specified. The negative tests show that duplicate slots, overlapping mechanic assignments, out-of-stock part issues, invalid values and orphan records are stopped by the database itself, and that stock deduction and invoice calculation produce the expected figures: (1100 + 3600) x 1.18 = 5546.00.

CHAPTER 4: CONCLUSION

This Project-Based Learning activity completed the design, normalization, implementation and application development of a Vehicle Service Centre Management System. Ten entities were derived from the requirements, modelled in an ER diagram and normalized to Third Normal Form. The key business rules, namely single booking per slot, one job card per booking, no overlapping mechanic assignments, no issue of parts beyond stock and one invoice per job card, are enforced inside the database through constraints and triggers. A Python application using parameterized DB-API queries delivers the full workflow from registration to invoice, with automatic stock deduction, GST-based billing and operational reports. The test suite confirmed the expected behaviour in all fifteen cases.

4.1 Limitations and Future Enhancements

The current implementation has the following limitations, which also point to future work:

1. Single-user terminal interface: a web or desktop front end would suit advisors and mechanics better.
2. Embedded SQLite database: for several branches and concurrent users the schema can be moved to MySQL or PostgreSQL, which needs the Review 2 DDL types, a different driver and %s placeholders.
3. Flat labor rate per task and a fixed 18% GST rate: labor by hours worked and configurable tax rates can be added.
4. No login or roles: role-based authentication for the four user roles is a natural next step.
5. No customer notifications or online payment: SMS or email reminders and a payment gateway can be integrated.

5. REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] J. D. Ullman and J. Widom, A First Course in Database Systems, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[5] SQLite Development Team, "SQLite Documentation," SQLite.org. [Online]. Available: https://www.sqlite.org/docs.html

[6] Python Software Foundation, "Python Database API Specification v2.0 (PEP 249)," Python.org. [Online]. Available: https://peps.python.org/pep-0249/

[7] Python Software Foundation, "sqlite3: DB-API 2.0 interface for SQLite databases," Python documentation. [Online]. Available: https://docs.python.org/3/library/sqlite3.html

[8] ISO/IEC, "Information technology - Database languages - SQL," ISO/IEC 9075, Geneva, Switzerland.

6. APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete DDL and trigger script is part of the application source file service_centre.py (constant SCHEMA) and is reproduced below.

```
PRAGMA foreign_keys = ON;
CREATE TABLE IF NOT EXISTS Customer (
  CustomerID INTEGER PRIMARY KEY AUTOINCREMENT,
  Name TEXT NOT NULL, Phone TEXT NOT NULL UNIQUE, Email TEXT UNIQUE, Address TEXT);
CREATE TABLE IF NOT EXISTS Vehicle (
  RegNo TEXT PRIMARY KEY,
  CustomerID INTEGER NOT NULL REFERENCES Customer(CustomerID) ON DELETE CASCADE,
  Make TEXT NOT NULL, Model TEXT NOT NULL,
  MfgYear INTEGER CHECK (MfgYear BETWEEN 1990 AND 2030),
  FuelType TEXT CHECK (FuelType IN ('Petrol','Diesel','CNG','Electric')));
CREATE TABLE IF NOT EXISTS Service_Booking (
  BookingID INTEGER PRIMARY KEY AUTOINCREMENT,
  RegNo TEXT NOT NULL REFERENCES Vehicle(RegNo),
  BookingDate TEXT NOT NULL, SlotTime TEXT NOT NULL,
  Status TEXT DEFAULT 'Booked'
    CHECK (Status IN ('Booked','Checked-In','Completed','Cancelled')),
  UNIQUE (RegNo, BookingDate, SlotTime));
CREATE TABLE IF NOT EXISTS Job_Card (
  JobCardID INTEGER PRIMARY KEY AUTOINCREMENT,
  BookingID INTEGER NOT NULL UNIQUE REFERENCES Service_Booking(BookingID),
  DateIn TEXT NOT NULL, DateOut TEXT,
  JobStatus TEXT DEFAULT 'Open' CHECK (JobStatus IN ('Open','In Progress','Closed')),
  CHECK (DateOut IS NULL OR DateOut >= DateIn));
CREATE TABLE IF NOT EXISTS Mechanic (
  MechanicID INTEGER PRIMARY KEY AUTOINCREMENT,
  Name TEXT NOT NULL, Specialization TEXT, Phone TEXT UNIQUE);
CREATE TABLE IF NOT EXISTS Task (
  TaskID INTEGER PRIMARY KEY AUTOINCREMENT,
  TaskName TEXT NOT NULL UNIQUE, LaborRate REAL CHECK (LaborRate >= 0));
CREATE TABLE IF NOT EXISTS Assignment (
  AssignmentID INTEGER PRIMARY KEY AUTOINCREMENT,
  JobCardID INTEGER NOT NULL REFERENCES Job_Card(JobCardID),
  TaskID INTEGER NOT NULL REFERENCES Task(TaskID),
  MechanicID INTEGER NOT NULL REFERENCES Mechanic(MechanicID),
  StartTime TEXT NOT NULL, EndTime TEXT, Status TEXT DEFAULT 'Assigned',
  CHECK (EndTime IS NULL OR EndTime > StartTime));
CREATE TABLE IF NOT EXISTS Spare_Part (
  PartID INTEGER PRIMARY KEY AUTOINCREMENT,
  PartName TEXT NOT NULL UNIQUE, UnitPrice REAL CHECK (UnitPrice > 0),
  StockQty INTEGER NOT NULL DEFAULT 0 CHECK (StockQty >= 0),
  ReorderLevel INTEGER DEFAULT 5);
CREATE TABLE IF NOT EXISTS Part_Usage (
  UsageID INTEGER PRIMARY KEY AUTOINCREMENT,
  JobCardID INTEGER NOT NULL REFERENCES Job_Card(JobCardID),
  PartID INTEGER NOT NULL REFERENCES Spare_Part(PartID),
  Quantity INTEGER NOT NULL CHECK (Quantity > 0),
  BilledUnitPrice REAL NOT NULL);
CREATE TABLE IF NOT EXISTS Invoice (
  InvoiceID INTEGER PRIMARY KEY AUTOINCREMENT,
  JobCardID INTEGER NOT NULL UNIQUE REFERENCES Job_Card(JobCardID),
  LaborTotal REAL DEFAULT 0, PartsTotal REAL DEFAULT 0, NetAmount REAL DEFAULT 0,
  PaymentStatus TEXT DEFAULT 'Unpaid');


CREATE TRIGGER IF NOT EXISTS trg_no_overlap
BEFORE INSERT ON Assignment
WHEN NEW.EndTime IS NOT NULL AND EXISTS (
  SELECT 1 FROM Assignment a
  WHERE a.MechanicID = NEW.MechanicID AND a.EndTime IS NOT NULL
    AND NEW.StartTime < a.EndTime AND NEW.EndTime > a.StartTime)
BEGIN SELECT RAISE(ABORT, 'Mechanic already booked in that time slot'); END;


CREATE TRIGGER IF NOT EXISTS trg_check_stock
BEFORE INSERT ON Part_Usage
WHEN (SELECT StockQty FROM Spare_Part WHERE PartID = NEW.PartID) < NEW.Quantity
BEGIN SELECT RAISE(ABORT, 'Insufficient stock'); END;


CREATE TRIGGER IF NOT EXISTS trg_deduct_stock
AFTER INSERT ON Part_Usage
BEGIN UPDATE Spare_Part SET StockQty = StockQty - NEW.Quantity
      WHERE PartID = NEW.PartID; END;
```

APPENDIX B - APPLICATION SOURCE CODE

The complete source code is archived in service_centre.py. The core functions are shown below.

```
def connect(path=DB_FILE):
    con = sqlite3.connect(path)
    con.execute("PRAGMA foreign_keys = ON")
    con.executescript(SCHEMA)
    return con


def run(con, label, sql, args=()):
    """Execute one write; report success or the database's own error message."""
    try:
        cur = con.execute(sql, args); con.commit()
        print(f"[OK]      {label}"); return cur
    except sqlite3.Error as e:
        con.rollback()
        msg = "Duplicate entry rejected (UNIQUE constraint)" if "UNIQUE" in str(e) else e
        print(f"[BLOCKED] {label} -> {msg}"); return None


def assign_task(con, job, task, mech, start, end):
    return run(con, f"Assign task {task} to mechanic {mech} ({start[-5:]}-{end[-5:]})",
               "INSERT INTO Assignment (JobCardID,TaskID,MechanicID,StartTime,EndTime) "
               "VALUES (?,?,?,?,?)", (job, task, mech, start, end))


def use_part(con, job, part, qty):
    return run(con, f"Use part {part} x{qty} on job card {job}",
               "INSERT INTO Part_Usage (JobCardID,PartID,Quantity,BilledUnitPrice) "
               "SELECT ?,PartID,?,UnitPrice FROM Spare_Part WHERE PartID=?", (job, qty, part))


def generate_invoice(con, job, date):
    labor = con.execute("SELECT IFNULL(SUM(t.LaborRate),0) FROM Assignment a "
                        "JOIN Task t ON t.TaskID=a.TaskID WHERE a.JobCardID=?", (job,)).fetchone()[0]
    parts = con.execute("SELECT IFNULL(SUM(Quantity*BilledUnitPrice),0) FROM Part_Usage "
                        "WHERE JobCardID=?", (job,)).fetchone()[0]
    net = round((labor + parts) * (1 + GST), 2)
    c = run(con, f"Generate invoice for job card {job} (labor {labor:.0f} + parts {parts:.0f} + 18% GST)",
            "INSERT INTO Invoice (JobCardID,LaborTotal,PartsTotal,NetAmount) VALUES (?,?,?,?)",
            (job, labor, parts, net))
    if c:
        con.execute("UPDATE Job_Card SET JobStatus='Closed', DateOut=? WHERE JobCardID=?", (date, job))
        con.execute("UPDATE Service_Booking SET Status='Completed' WHERE BookingID="
                    "(SELECT BookingID FROM Job_Card WHERE JobCardID=?)", (job,))
        con.commit()
```

APPENDIX C - DEMONSTRATION OUTPUT AND TEST DATA

The output of the scripted demonstration (python service_centre.py --demo) is reproduced below. The same run is archived in demo_output.txt.

```
--- Step 1-3: register, book, check in ---
[OK]      Register customer Ravi Kumar
[OK]      Register vehicle TS09AB1234
[OK]      Book TS09AB1234 on 2026-10-12 10:00
[BLOCKED] Book TS09AB1234 on 2026-10-12 10:00 -> Duplicate entry rejected (UNIQUE constraint)
[OK]      Check in booking 1
[OK]      Open job card for booking 1


--- Step 4: schedule mechanics (overlap trigger) ---
[OK]      Assign task 1 to mechanic 1 (10:15-10:45)
[BLOCKED] Assign task 2 to mechanic 1 (10:30-11:30) -> Mechanic already booked in that time slot
[OK]      Assign task 2 to mechanic 1 (11:00-12:00)


--- Step 5: spare parts (stock trigger) ---
[OK]      Use part 1 x4 on job card 1
[BLOCKED] Use part 2 x5 on job card 1 -> Insufficient stock
[OK]      Use part 2 x1 on job card 1


--- Step 6: billing ---
[OK]      Generate invoice for job card 1 (labor 1100 + parts 3600 + 18% GST)


--- Step 7: reports ---


== Job cards ==
JobCardID  RegNo       JobStatus  DateIn      DateOut
```

```
---------  ----------  ---------  ----------  ----------
1          TS09AB1234  Closed     2026-10-12  2026-10-12


== Low stock parts ==
PartName       StockQty  ReorderLevel
-------------  --------  ------------
Brake Pad Set  2         5


== Mechanic workload ==
Name        Tasks  Hours
----------  -----  -----
Imran Khan  2      1.50


== Unpaid invoices ==
InvoiceID  Name        NetAmount
---------  ----------  ---------
1          Ravi Kumar  5546.00


== Monthly revenue ==
Month    Revenue
-------  -------
2026-10  5546.00
[OK]      Mark invoice 1 paid


== Unpaid invoices ==
InvoiceID  Name  NetAmount
---------  ----  ---------
(no rows)
```